

Annexe B : Éléments techniques de l'application

```

services:
  n8n:
    image: n8nio/n8n:latest
    container_name: n8n
    restart: always
    ports:
      - "5678:5678"
    environment:
      - N8N_PORT=5678
      - NODE_ENV=production
      - WEBHOOK_URL=http://sandpaper-steerable-blandness.ngrok-free.dev/
      - N8N_SECURE_COOKIE=false
      - NODES_EXCLUDE=[]
      - GENERIC_TIMEZONE=Africa/Tunis
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=redis
      - QUEUE_BULL_REDIS_PORT=6379
      - N8N_RESTRICT_FILE_ACCESS_T0=/data/shared
      - BRIDGE_URL=https://retrace-font-implicit.ngrok-free.dev
    volumes:
      - ~/.n8n:/home/node/.n8n
      - ./shared:/data/shared
      - ./home/node/soc-project
    depends_on:
      - redis

  n8n-worker:
    image: n8nio/n8n:latest
    restart: always
    command: worker
    environment:
      - N8N_PROTOCOL=http
      - NODE_ENV=production
      - GENERIC_TIMEZONE=Africa/Tunis
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=redis
      - QUEUE_BULL_REDIS_PORT=6379
      - N8N_RESTRICT_FILE_ACCESS_T0=/data/shared
    depends_on:

```

FIGURE 1 – Configuration Docker Compose – Partie 1

```

    - redis

n8n-worker:
  image: n8nio/n8n:latest
  restart: always
  command: worker
  environment:
    - N8N_PROTOCOL=http
    - NODE_ENV=production
    - GENERIC_TIMEZONE=Africa/Tunis
    - EXECUTIONS_MODE=queue
    - QUEUE_BULL_REDIS_HOST=redis
    - QUEUE_BULL_REDIS_PORT=6379
    - N8N_RESTRICT_FILE_ACCESS_T0=/data/shared
  depends_on:
    - redis
    - n8n
  volumes:
    - ~/.n8n:/home/node/.n8n
    - ./shared:/data/shared
    - ./home/node/soc-project

  deploy:
    replicas: 3

redis:
  image: redis:7-alpine
  container_name: n8n-redis
  restart: always
  volumes:
    - redis_data:/data
edge-tts:
  build:
    context: ./edge-tts-service
  container_name: edge-tts
  restart: always
  ports:
    - "5050:5050"
faster-whisper:
  image: onerahmet/openai-whisper-asr-webservice:latest
  container_name: faster-whisper

```

FIGURE 2 – Configuration Docker Compose – Partie 2

```

faster-whisper:
  image: onerahmet/openai-whisper-asr-webservice:latest
  container_name: faster-whisper
  restart: always
  environment:
    - ASR_MODEL=base
    - ASR_ENGINE=faster_whisper
  ports:
    - "9000:9000"
  volumes:
    - whisper_cache:/root/.cache/
ollama:
  image: ollama/ollama:latest
  container_name: ollama
  restart: always
  ports:
    - "11434:11434"
  volumes:
    - ollama_data:/root/.ollama

```

FIGURE 3 – Configuration Docker Compose – Partie 3

```

volumes:
  redis_data:
  whisper_cache:
  ollama_data:

```

FIGURE 4 – Configuration Docker Compose – Partie 4

```

import asyncio
import io
import os
import struct
import wave

import aiohttp
from aiohttp import web

from audiosocket_server import Call, serve as audiosocket_serve

N8N_BASE_URL = os.environ.get("N8N_BASE_URL", "http://localhost:9500")
CALL_START_URL = f"{N8N_BASE_URL}/webhook/3cx-bridge/call-start"
AUDIO_CHUNK_URL = f"{N8N_BASE_URL}/webhook/3cx-bridge/audio"
CALL_END_URL = f"{N8N_BASE_URL}/webhook/3cx-bridge/call-end"

REGISTER_PORT = int(os.environ.get("REGISTER_PORT", "8090"))
AUDIO_SOCKET_PORT = int(os.environ.get("AUDIO_SOCKET_PORT", "8091"))

# One frame = 20ms of audio (320 bytes @ 8kHz/16-bit/mono).
FRAME_BYTES = 320

# Safety cap: if the caller talks continuously with no pause, force a
# flush after this many frames so we don't buffer forever. Not a "turn
# end" - just a chunking safety valve. 1500 frames = 30s.
CHUNK_MAX_FRAMES = 1500

# A "turn" is considered complete once the caller has been silent for
# this many consecutive frames. 500 frames * 20ms = 10 seconds, per
# spec: the bot should only respond after ~10s of silence.
SILENCE_RMS_THRESHOLD = 400
SILENCE_FRAMES_TO_CUT = 500

pending_call_metadata = {}

# Registry of calls currently in progress, keyed by call_id (uuid).
# Populated in handle_call() as soon as the uuid is known, removed when
# the call ends. This is what lets an HTTP request to /speak find the
# right live AudioSocket connection to write into.
active_calls = {}
resolved_calls = set() # call_ids the bot successfully handled

```

FIGURE 5 – Code source bridge.py – Partie 1

```

def frame_rms(frame: bytes) -> int:
    if len(frame) < 2:
        return 0
    samples = struct.unpack(f"<{len(frame)//2}h", frame)
    return int((sum(s * s for s in samples) / len(samples)) ** 0.5)

def trim_silence(frames, threshold=SILENCE_RMS_THRESHOLD, pad_frames=10):
    """
    Drop leading/trailing frames that are pure silence, keeping a small
    padding around the real speech. This concentrates the actual signal
    instead of sending Whisper a mostly-empty buffer (e.g. 2s of real
    speech buried in a 10s chunk), which was hurting transcription
    accuracy even with a better model.
    """
    first_speech = None
    last_speech = None
    for i, f in enumerate(frames):
        if frame_rms(f) >= threshold:
            if first_speech is None:
                first_speech = i
            last_speech = i
    if first_speech is None:
        return frames # nothing but silence - leave as-is, caller handles this case
    start = max(0, first_speech - pad_frames)
    end = min(len(frames), last_speech + pad_frames + 1)
    return frames[start:end]

def frames_to_wav_bytes(frames, sample_rate=8000):
    import audioop
    frames = trim_silence(frames)
    raw = b"".join(frames)

    # Boost/normalize volume so quiet audio is more audible/detectable.
    # audioop.max returns the peak absolute sample value in the buffer.
    peak = audioop.max(raw, 2)
    if peak > 0:
        # Target peak at ~80% of int16 range (26214), cap gain at 10x
        # so we don't blow up noise floor on truly silent audio.
        target_peak = 26214
        gain = min(target_peak / peak, 4.0)
        if gain > 1.05:

```

FIGURE 6 – Code source bridge.py – Partie 2

```

        raw = audioop.mul(raw, 2, gain)

    buf = io.BytesIO()
    with wave.open(buf, "wb") as w:
        w.setnchannels(1)
        w.setsampwidth(2)
        w.setframerate(sample_rate)
        w.writeframes(raw)
    return buf.getvalue()

async def handle_register_call(request):
    data = await request.post()
    uuid_ = data.get("uuid")
    if not uuid_:
        return web.json_response({"error": "missing uuid"}, status=400)
    uuid_ = uuid_.replace("-", "")
    pending_call_metadata[uuid_] = {
        "ani": data.get("ani", ""),
        "did": data.get("did", ""),
        "dtmf": data.get("dtmf", "")
    }
    print(f"[bridge] Registered call metadata for {uuid_}: {pending_call_metadata[uuid_]}")
    return web.json_response({"status": "registered"})

async def convert_to_pcm(input_bytes: bytes) -> bytes:
    """
    Run input audio bytes (mp3, wav, whatever) through ffmpeg and get back
    raw 8kHz, 16-bit, mono, little-endian PCM - the format AudioSocket/
    Asterisk expects here. Runs ffmpeg as a subprocess, piping bytes in
    and out over stdin/stdout so nothing touches disk.
    """
    proc = await asyncio.create_subprocess_exec(
        "ffmpeg",
        "-i", "pipe:0",
        "-ar", "8000",
        "-ac", "1",
        "-f", "s16le",
        "-acodec", "pcm_s16le",
        "pipe:1",
        stdin=asyncio.subprocess.PIPE,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )

```

FIGURE 7 – Code source bridge.py – Partie 3

```

    )
    stdout, stderr = await proc.communicate(input=input_bytes)
    if proc.returncode != 0:
        raise RuntimeError(f"ffmpeg failed (code {proc.returncode}): {stderr.decode(errors='replace')[:500]}")
    return stdout

async def handle_speak(request):
    """
    POST /calls/{call_id}/speak
    Expects the finished TTS audio as the raw request body (any format
    ffmpeg can decode - mp3 from edge-tts is fine), with Content-Type
    set appropriately (e.g. audio/mpeg). n8n should call edge-tts's
    own /tts endpoint first, then forward the resulting audio bytes here.
    """
    call_id = request.match_info["call_id"].replace("-", "")

    call = active_calls.get(call_id)
    if call is None:
        return web.json_response(
            {"error": f"no active call for call_id={call_id}"}, status=404
        )

    audio_bytes = await request.read()
    if not audio_bytes:
        return web.json_response({"error": "empty request body"}, status=400)

    try:
        pcm_bytes = await convert_to_pcm(audio_bytes)
    except Exception as e:
        print(f"[bridge] /speak ffmpeg conversion failed: {e!r}")
        return web.json_response({"error": f"conversion failed: {e}"}, status=500)

    try:
        await call.send_audio(pcm_bytes)
    except Exception as e:
        print(f"[bridge] /speak send_audio failed: {e!r}")
        return web.json_response({"error": f"send failed: {e}"}, status=500)

    print(f"[bridge] /speak: sent {len(pcm_bytes)} bytes of PCM to call {call_id}")
    return web.json_response({"status": "spoken", "call_id": call_id, "bytes_sent": len(pcm_bytes)})

async def handle_mark_resolved(request):
    """
    POST /calls/{call_id}/mark-resolved
    """
    call_id = request.match_info["call_id"].replace("-", "")
    try:
        body = await request.json()
    except Exception:
        body = {}
    resolved = body.get("resolved", True)
    if resolved:
        resolved_calls.add(call_id)
        print(f"[bridge] Call {call_id} marked resolved=True")
    else:
        resolved_calls.discard(call_id)
        print(f"[bridge] Call {call_id} marked resolved=False")
    return web.json_response({"status": "ok", "call_id": call_id, "resolved": resolved})

def build_http_app():
    app = web.Application(client_max_size=20 * 1024 * 1024) # allow decent-sized audio bodies
    app.router.add_post("/register-call", handle_register_call)
    app.router.add_post("/calls/{call_id}/speak", handle_speak)
    app.router.add_post("/calls/{call_id}/mark-resolved", handle_mark_resolved)
    return app

async def post_call_start(session, call_id, meta):
    body = {
        "call_id": call_id,
        "caller_phone": meta.get("anl", ""),
        "caller_name": "",
        "dtmf_choice": meta.get("dtmf", "")
    }
    try:
        async with session.post(CALL_START_URL, json=body, timeout=30) as resp:
            print(f"[bridge] call-start POSTed, status={resp.status}")
    except Exception as e:
        print(f"[bridge] call-start POST failed: {e!r}")

async def post_audio_chunk(session, call_id, sequence, wav_bytes, is_final):
    form = aiohttp.FormData()
    form.add_field("data", wav_bytes, filename=f"chunk-{sequence}.wav", content_type="audio/wav")
    async with session.post(CALL_CHUNK_URL, data=form, timeout=30) as resp:
        print(f"[bridge] call-chunk POSTed, status={resp.status}")

```

FIGURE 8 – Code source bridge.py – Partie 4

```

    return web.json_response({"status": "spoken", "call_id": call_id, "bytes_sent": len(pcm_bytes)})

async def handle_mark_resolved(request):
    call_id = request.match_info["call_id"].replace("-", "")
    try:
        body = await request.json()
    except Exception:
        body = {}
    resolved = body.get("resolved", True)
    if resolved:
        resolved_calls.add(call_id)
        print(f"[bridge] Call {call_id} marked resolved=True")
    else:
        resolved_calls.discard(call_id)
        print(f"[bridge] Call {call_id} marked resolved=False")
    return web.json_response({"status": "ok", "call_id": call_id, "resolved": resolved})

def build_http_app():
    app = web.Application(client_max_size=20 * 1024 * 1024) # allow decent-sized audio bodies
    app.router.add_post("/register-call", handle_register_call)
    app.router.add_post("/calls/{call_id}/speak", handle_speak)
    app.router.add_post("/calls/{call_id}/mark-resolved", handle_mark_resolved)
    return app

async def post_call_start(session, call_id, meta):
    body = {
        "call_id": call_id,
        "caller_phone": meta.get("anl", ""),
        "caller_name": "",
        "dtmf_choice": meta.get("dtmf", "")
    }
    try:
        async with session.post(CALL_START_URL, json=body, timeout=30) as resp:
            print(f"[bridge] call-start POSTed, status={resp.status}")
    except Exception as e:
        print(f"[bridge] call-start POST failed: {e!r}")

async def post_audio_chunk(session, call_id, sequence, wav_bytes, is_final):
    form = aiohttp.FormData()
    form.add_field("data", wav_bytes, filename=f"chunk-{sequence}.wav", content_type="audio/wav")
    async with session.post(CALL_CHUNK_URL, data=form, timeout=30) as resp:
        print(f"[bridge] call-chunk POSTed, status={resp.status}")

```

FIGURE 9 – Code source bridge.py – Partie 5

```

params = {
    "call_id": call_id,
    "sequence": str(sequence),
    "is_final": "true" if is_final else "false",
}
try:
    async with session.post(AUDIO_CHUNK_URL, data=form, params=params, timeout=60) as resp:
        print(f"[bridge] audio-chunk #{sequence} POSTed (final={is_final}), status={resp.status}")
except Exception as e:
    # Don't let a slow/failed transcription kill the whole call handler -
    # losing one chunk shouldn't mean we never report call-end.
    print(f"[bridge] audio-chunk #{sequence} POST failed: {e}")

async def post_call_end(session, call_id, meta, bot_handled=False, transcript=""):
    body = {
        "call_id": call_id,
        "caller_phone": meta.get("ani", ""),
        "caller_name": "",
        "bot_handled": bot_handled,
        "transcript": transcript
    }
    try:
        async with session.post(CALL_END_URL, json=body, timeout=30) as resp:
            print(f"[bridge] call-end POSTed, status={resp.status}")
    except Exception as e:
        print(f"[bridge] call-end POST failed: {e}")

async def handle_call(call: Call):
    async with aiohttp.ClientSession() as session:
        buffer = []
        silence_run = 0
        sequence = 0
        call_id = None
        meta = {}
        had_speech = False

        try:
            async for frame in call.read_audio_frames():
                if call.uuid and call_id is None:
                    call_id = call.uuid
                    meta = pending_call_metadata.pop(call_id, {})
                    active_calls[call_id] = call
                    print(f"[bridge] Call {call_id} audio stream started, metadata={meta}")
                    await post_call_start(session, call_id, meta)

```

FIGURE 10 – Code source bridge.py – Partie 6

```

    print(f"[bridge] Call {call_id} audio stream started, metadata={meta}")
    await post_call_start(session, call_id, meta)

    if call_id is None:
        continue

    rms = frame_rms(frame)
    buffer.append(frame)

    if rms < SILENCE_RMS_THRESHOLD:
        silence_run += 1
    else:
        silence_run = 0
        had_speech = True

    turn_complete = (
        had_speech
        and silence_run >= SILENCE_FRAMES_TO_CUT
        and len(buffer) > SILENCE_FRAMES_TO_CUT
    )
    hit_safety_cap = len(buffer) >= CHUNK_MAX_FRAMES
    pure_silence_timeout = (
        not had_speech
        and silence_run >= SILENCE_FRAMES_TO_CUT
        and len(buffer) > SILENCE_FRAMES_TO_CUT
    )
    should_flush = turn_complete or hit_safety_cap or pure_silence_timeout

    if should_flush:
        if pure_silence_timeout and not hit_safety_cap:
            # Nothing but dead air the whole window - drop it
            # silently, don't send it as a "turn" (this is what
            # was causing the bot to repeat itself every ~10s
            # of silence with no new input from the caller).
            print(f"[bridge] dropping {len(buffer)} frames of pure silence (no turn)")
            buffer = []
            silence_run = 0
            had_speech = False
        else:
            wav_bytes = frames_to_wav_bytes(buffer)
            sequence += 1
            print(f"[bridge] flushing {len(buffer)} frames -> {len(wav_bytes)} bytes wav "
                  f"(turn_complete={turn_complete}, hit_safety_cap={hit_safety_cap})")
            # Treat as a real turn (worth a bot response) whenever
            # the chunk actually contains speech - whether it flushed

```

FIGURE 11 – Code source bridge.py – Partie 7

```

        else:
            wav_bytes = frames_to_wav_bytes(buffer)
            sequence += 1
            print(f"[bridge] flushing {len(buffer)} frames -> {len(wav_bytes)} bytes wav "
                  f"(turn_complete={turn_complete}, hit_safety_cap={hit_safety_cap})")
            # Treat as a real turn (worth a bot response) whenever
            # the chunk actually contains speech - whether it flushed
            # from a genuine 10s pause OR from hitting the safety cap
            # while still talking. Only pure silence (handled above)
            # is excluded.
            is_final_turn = turn_complete or (hit_safety_cap and had_speech)
            await post_audio_chunk(session, call_id, sequence, wav_bytes, is_final=is_final_turn)
            buffer = []
            silence_run = 0
            had_speech = False

    if call_id:
        if buffer:
            wav_bytes = frames_to_wav_bytes(buffer)
            sequence += 1
            print(f"[bridge] flushing {len(buffer)} frames -> {len(wav_bytes)} bytes wav (call ended)")
            await post_audio_chunk(session, call_id, sequence, wav_bytes, is_final=True)
            bot_handled = call_id in resolved_calls
            resolved_calls.discard(call_id)
            await post_call_end(session, call_id, meta, bot_handled=bot_handled)
        finally:
            if call_id:
                active_calls.pop(call_id, None)

async def main():
    runner = web.AppRunner(build_http_app())
    await runner.setup()
    site = web.TCPSite(runner, "0.0.0.0", REGISTER_PORT)
    await site.start()
    print(f"[bridge] Registration HTTP server listening on :{REGISTER_PORT}")

    await audiosocket_serve("0.0.0.0", AUDIOSOCKET_PORT, handle_call)

if __name__ == "__main__":
    asyncio.run(main())

```

FIGURE 12 – Code source `bridge.py` – Partie 8

```

import asyncio
import struct
from typing import Awaitable, Callable

FRAME_BYTES = 320 # 20ms @ 8kHz, 16-bit mono
TYPE_TERMINATE = 0x00
TYPE_UUID = 0x01
TYPE_AUDIO = 0x10

class Call:
    def __init__(self, reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
        self.reader = reader
        self.writer = writer
        self.uuid = None
        self.closed = False
        self.frames_received = 0
        self.bytes_received = 0

    async def read_audio_frames(self):
        while not self.closed:
            try:
                header = await self.reader.readexactly(3)
            except (asyncio.IncompleteReadError, ConnectionResetError):
                break
            pkt_type, length = header[0], struct.unpack(">H", header[1:3])[0]
            payload = await self.reader.readexactly(length) if length else b""
            print(f"[audiosocket] packet type={pkt_type} length={length}")
            if pkt_type == TYPE_UUID:
                self.uuid = payload.hex()
                print(f"[audiosocket] UUID set: {self.uuid}")
            elif pkt_type == TYPE_AUDIO:
                self.frames_received += 1
                self.bytes_received += len(payload)
                yield payload
            elif pkt_type == TYPE_TERMINATE:
                break
        self.closed = True

    async def send_audio(self, pcm_bytes: bytes):
        if self.closed:
            return
        # Pace frames at real-time (20ms per frame) - AudioSocket/Asterisk
        # expects audio arriving at real playback speed, blasting all

```

FIGURE 13 – Configuration d'AudioSocket – Partie 1

```

# expects audio arriving at real playback speed. Blasting all
# frames instantly overflows Asterisk's buffer and truncates
# playback after whatever fit in that window.
frame_duration = 0.02 # 20ms per 320-byte frame @ 8kHz/16-bit
for i in range(0, len(pcm_bytes), FRAME_BYTES):
    if self.closed:
        break
    chunk = pcm_bytes[i:i + FRAME_BYTES]
    if len(chunk) < FRAME_BYTES:
        chunk += b"\x00" * (FRAME_BYTES - len(chunk))
    self.writer.write(bytes([TYPE_AUDIO]) + struct.pack(">H", len(chunk)) + chunk)
    await self.writer.drain()
    await asyncio.sleep(frame_duration)

async def hangup(self):
    if self.closed:
        return
    self.closed = True
    try:
        self.writer.write(bytes([TYPE_TERMINATE]) + struct.pack(">H", 0))
        await self.writer.drain()
    except Exception:
        pass
    self.writer.close()

async def serve(host: str, port: int, on_call: Callable[[Call], Awaitable[None]]):
    async def handle(reader, writer):
        peer = writer.get_extra_info("peername")
        print(f"[audiosocket] connection accepted from {peer}")
        call = Call(reader, writer)
        try:
            await on_call(call)
        except Exception as e:
            print(f"[audiosocket] call handler error: {e!r}")
        finally:
            print(f"[audiosocket] connection closed (uuid={call.uuid})")
            await call.hangup()

    server = await asyncio.start_server(handle, host, port)
    print(f"AudioSocket server listening on {host}:{port}")
    async with server:
        await server.serve_forever()

```

FIGURE 14 – Configuration d'AudioSocket – Partie 2